

Collision-Risk Triage: Frontend Code

TechFest 2026-27 Space Technology Hackathon | Team Jugaad Junction

What this PDF contains

The React dashboard (Vite + Tailwind CSS + Recharts): a red/amber/green alert table, a "Why this alert?" box with Escalate / Monitor / Dismiss buttons, a score chart, and the what-if uncertainty slider. It talks to the FastAPI backend at `http://127.0.0.1:8000`, so the backend must be running.

Folder layout

```
space-triage/  
  frontend/  
    package.json      created by Vite  
    vite.config.js    File 1 (replace contents)  
    src/  
      index.css       File 2 (replace contents)  
      App.jsx         File 3 (replace contents)  
      App.css         delete this file
```

Step 0: Create the React project

```
cd %HOME%\Documents\space-triage  
npm create vite@latest frontend
```

Answer the questions: install create-vite = **y**, framework = **React**, variant = **JavaScript**, linter = press Enter, rolldown / install and start now = **No**. Then:

```
cd frontend  
npm install  
npm install tailwindcss @tailwindcss/vite recharts
```

Your prompt must end with `\space-triage\frontend>` before running the installs. This needs internet and takes a few minutes.

File 1: vite.config.js

Run `notepad vite.config.js`, press Ctrl+A, Delete, paste this, Ctrl+S, close.

```
import { defineConfig } from 'vite'  
import react from '@vitejs/plugin-react'  
import tailwindcss from '@tailwindcss/vite'  
  
export default defineConfig({  
  plugins: [react(), tailwindcss()],  
})
```

File 2: src\index.css

Run `notepad src\index.css`, replace everything with this single line, save, close. Then delete the leftover style file (an error saying it does not exist is fine).

```
@import "tailwindcss";  
  
del src\App.css
```

File 3: src\App.jsx (the dashboard)

Run notepad src\App.jsx, press Ctrl+A, Delete, paste the code below, Ctrl+S, close. Make sure the file name is exactly App.jsx and not App.jsx.txt.

```
import { useEffect, useState } from "react";
import { BarChart, Bar, XAxis, YAxis, Tooltip, ResponsiveContainer } from "recharts";

const API = "http://127.0.0.1:8000";
const color = { High: "bg-red-600", Medium: "bg-amber-500", Low: "bg-green-600" };

export default function App() {
  const [alerts, setAlerts] = useState([]);
  const [selId, setSelId] = useState(null);
  const [mult, setMult] = useState(1);

  async function load() {
    const res = await fetch(`${API}/alerts?sigma_mult=${mult}`);
    setAlerts(await res.json());
  }
  useEffect(() => { load(); }, [mult]);

  async function decide(id, action) {
    await fetch(`${API}/alerts/${id}/decision`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ action })
    });
    load();
  }

  const sel = alerts.find((a) => a.id === selId) || alerts[0];
  const top = alerts.slice(0, 10).map((a) => ({ name: a.object_id, score: a.score }));

  return (
    <div className="min-h-screen bg-slate-950 text-slate-100 p-6">
      <h1 className="text-2xl font-bold mb-4">Collision-Risk Triage</h1>

      <div className="mb-4 p-3 rounded bg-slate-900">
        <label>What-if: uncertainty x{mult.toFixed(1)}</label>
        <input type="range" min="0.5" max="3" step="0.1" value={mult}
          onChange={(e) => setMult(parseFloat(e.target.value))}
          className="w-full" />
      </div>

      <div className="grid md:grid-cols-2 gap-4">
        <div className="overflow-auto max-h-[70vh]">
          <table className="w-full text-sm">
            <thead>
              <tr className="text-left text-slate-400">
                <th>Tier</th><th>Object</th><th>TCA (h)</th><th>Miss (m)</th><th>Pc</th><th>Status</th>
              </tr>
            </thead>
            <tbody>
              {alerts.map((a) => (
                <tr key={a.id} onClick={() => setSelId(a.id)}
                  className="cursor-pointer hover:bg-slate-800">
                  <td><span className={`${px-2 py-1 rounded text-xs ${color[a.tier]}`}>{a.tier}</span></td>
                  <td>{a.object_id}</td>
                  <td>{a.tca_hours.toFixed(1)}</td>
                  <td>{a.miss_distance_m.toFixed(0)}</td>
                  <td>{a.pc.toExponential(1)}</td>
                  <td>{a.decision}</td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </div>
    </div>
  );
}
```

```

<div>
  {sel && (
    <div className="p-4 rounded bg-slate-900 mb-4">
      <h2 className="font-bold mb-2">Why this alert? ({sel.object_id})</h2>
      <p className="mb-3">{sel.why}</p>
      <div className="flex gap-2">
        [{"Escalate", "Monitor", "Dismiss"].map((act) => (
          <button key={act} onClick={() => decide(sel.id, act)}
            className="px-3 py-1 rounded bg-sky-600 hover:bg-sky-500">{act}</button>
        ))}
      </div>
    </div>
  )}
  <div className="h-64 p-2 rounded bg-slate-900">
    <ResponsiveContainer>
      <BarChart data={top}>
        <XAxis dataKey="name" tick={{ fontSize: 10 }} />
        <YAxis domain={[0, 1]} />
        <Tooltip />
        <Bar dataKey="score" fill="#38bdf8" />
      </BarChart>
    </ResponsiveContainer>
  </div>
</div>
</div>
</div>
);
}

```

Run the website

Window 1 must be running the backend:

```

cd $HOME\Documents\space-triage\backend
venv\Scripts\activate
uvicorn main:app --reload

```

Window 2 runs the frontend:

```

cd $HOME\Documents\space-triage\frontend
npm run dev

```

Open <http://localhost:5173> in Chrome or Edge. Test it: click a row (the "why" text changes), click Escalate (Status changes from Pending to Escalate), drag the slider (scores re-rank), and check <http://127.0.0.1:8000/audit> for the logged decision. Press Ctrl+C in each window to stop.

Common problems

Blank page	-> press F12, open Console, read the red text
Table is empty	-> backend (Window 1) is not running
No colors / plain styles	-> check vite.config.js and index.css, then restart npm run dev
npm run dev not found	-> you are not inside the frontend folder